

Platforma Big Data dla miejskich danych środowiskowych i transportowych

Autorzy: Jakub Kaproń, Miłosz Kita, Michał Kukla

1. Wstęp

Celem projektu jest opracowanie i wdrożenie architektury Big Data opartej na ekosystemie Apache, umożliwiającej integrację i analizę danych o warunkach pogodowych, jakości powietrza oraz lokalizacji autobusów miejskich w Warszawie w celu zbadania, w jaki sposób czynniki środowiskowe wpływają na opóźnienia komunikacji miejskiej.

Wybrane do analizy 3 źródła danych to:

- **Jakość Powietrza (<https://api.openaq.org/docs>):** Dane z globalnego repozytorium gromadzącego w czasie rzeczywistym i historycznie pomiary zanieczyszczeń (m.in. PM2.5, PM10, SO2, NO2, CO) z różnych stacji pomiarowych w Polsce. OpenAQ harmonizuje te dane, udostępniając je w jednolitym formacie JSON.
- **Bieżące lokalizacje autobusów komunikacji miejskiej (<https://api.um.warszawa.pl>):** Dane przedstawiają bieżące lokalizacje autobusów komunikacji miejskiej, udostępniane publicznie przez Miasto Stołeczne Warszawa. Aktualizują się co ok. 10 sekund i zwracają m.in. numer linii, współrzędne GPS, numer brygady oraz czas nadania pozycji. Dostęp wymaga bezpłatnego klucza API.
- **OpenWeather API (<https://openweathermap.org/current>):** dostarcza aktualne dane meteorologiczne z częstotliwością odświeżania co około 10 minut, w zależności od rodzaju usługi i lokalizacji. Dane obejmują m.in. temperaturę, wilgotność, prędkość wiatru, ciśnienie oraz warunki atmosferyczne. Prognozy są dostępne w interwałach 3-godzinnych na 5 dni do przodu. Serwis wykorzystuje globalną sieć stacji pomiarowych oraz modele numeryczne, aby zapewnić możliwie dokładne informacje dla dowolnego punktu na świecie.

Głównym celem analitycznym i biznesowym jest zbadanie wzajemnych zależności między warunkami atmosferycznymi i jakością powietrza a transportem miejskim, co ma wspierać podejmowanie strategicznych decyzji przez zarządców miast.

Architektura umożliwi:

- Badanie korelacji warunków środowiskowych (Pogoda i Jakość Powietrza) z wyborem środka transportu (mierzone pośrednio przez natężenie ruchu ZTM lub jego zakłócenia).
- Optymalizację operacyjną ZTM: Analiza, jak ekstremalna pogoda (np. opady śniegu, upały) wpływa na płynność ruchu i opóźnienia, co jest kluczowe dla zarządzania flotą i rozkładami jazdy.

Projekt ma na celu stworzenie skalowalnej platformy Big Data integrującej, przetwarzającej i analizującej miejskie dane środowiskowe oraz transportowe (z wykorzystaniem technologii Apache),

w celu przekształcenia surowych danych w użyteczne informacje wspierające podejmowanie decyzji w celu optymalizacji działania transportu publicznego i poprawy jakości życia w mieście.

2. Wymagania

Wymaganie (definicja)	Realizacja (Implementacja/ Konfiguracja)	Referencja w raporcie (Sekcja)
Pozyskanie danych z co najmniej trzech źródeł	1. Jakość Powietrza (OpenAQ API) - JSON, stacje Warszawy i okolic (https://api.openaq.org/docs) 2. Bieżące lokalizacje autobusów komunikacji miejskiej (https://api.um.warszawa.pl) - JSON 3. OpenWeather Weather API (https://openweathermap.org/current)	3. Opis architektury (I. Pozyskiwanie) 4. Źródła danych 5. Hadoop Batch Layer
Zaprojektowanie i wykonanie sposobu składowania danych (np. format, ustrukturyzowanie danych np. do postaci tabel Apache Hive)	Dane z trzech źródeł (OpenAQ, ZTM, OpenWeather) zostaną zapisane w ujednoliconym formacie Parquet w systemie plików HDFS. Następnie zostaną załadowane do tabel w Apache Hive, które będą partycjonowane według daty i typu źródła danych. Struktura tabel pozwoli na łatwe łączenie danych między zbiorami oraz późniejszą analizę w Apache Spark.	3. Opis architektury (III. Warstwa Wsadowa) 5. Hadoop Batch Layer
Zaprojektowanie i oprogramowanie komponentów służących do przetwarzania danych (np. konwersja formatów, filtrowanie, analiza)	Dane będą przetwarzane w przepływach Apache NiFi - m.in. konwersja z formatu JSON lub CSV do Parquet, usuwanie duplikatów, ujednolicenie nazw pól oraz dodawanie znaczników czasu i końcowo dodania danych do tabel w Hive. Apache Spark zostanie użyty do dalszego filtrowania i łączenia danych po kluczach wspólnych (np. lokalizacja, data). Wynikiem będzie zestaw danych gotowy do analizy korelacji między jakością powietrza, pogodą i opóźnieniami autobusów.	3. Opis architektury (I. Pozyskiwanie, IV. Wsadowa Analiza)

Zastosowanie środowisk Big Data do automatyzacji przepływu danych (co najmniej Apache NiFi)	Przepływ danych w Apache NiFi realizuje automatyczne pobieranie danych z kilku API, ich przetwarzanie i ujednolicenie, a następnie zapis do HDFS. Całość jest w pełni zautomatyzowana i monitorowana w interfejsie NiFi.	3. Opis architektury (I. Pozyskiwanie)
Zastosowanie środowisk Big Data do składowania danych (co najmniej jedno narzędzie z każdej z grup) - [Hadoop lub Hive] i [HBase, Cassandra, MongoDB]	Batch layer: dane surowe i przetworzone zostaną zapisane w HDFS i Apache Hive w strukturze tabelarycznej. Serving layer: agregaty i widoki wsadowe będą przechowywane w Apache HBase, umożliwiając szybkie odczyty.	3. Opis architektury (III. Warstwa Wsadowa, V. Warstwa Udostępniania)
Zastosowanie środowisk Big Data do wsadowej analizy danych (co najmniej Apache Spark)	Apache Spark zostanie wykorzystany do analizy danych zapisanych w Hive i HDFS. W ramach zadań wsadowych Spark obliczy m.in. średnie opóźnienia autobusów w zależności od stężenia zanieczyszczeń oraz warunków pogodowych. Wyniki analizy zostaną zapisane w nowych tabelach w Hive i HBase.	3. Opis architektury (IV. Wsadowa Analiza) 6. Dane Docelowe
Buforowanie danych na dowolnym etapie (Apache Kafka)	Zastosowanie Apache Kafka jako bufora danych (Message Broker) między modułem pozyskiwania (NiFi) a modułem składowania/przetwarzania. Ma na celu separację systemów oraz obsługę dużego wolumenu danych w czasie zbliżonym do rzeczywistego (np. z lokalizacji autobusów) i zapewnienie ich niezawodności.	3. Opis architektury (II. Buforowanie Danych)

3. Opis architektury

System oparto na architekturze typu Big Data, wykorzystując ekosystem Apache. Architektura dzieli się na warstwę wsadową (Batch Layer) oraz warstwę udostępniania (Serving Layer).

I. Pozyskiwanie i Wstępne Przetwarzanie (Ingestion)

- Narzędzie: Apache NiFi.
- Rola: Realizacja automatycznych przepływów danych (Flows). NiFi cyklicznie pobiera dane z trzech API: OpenAQ (Jakość Powietrza), API Miasta Stołecznego Warszawy (Lokalizacje Autobusów) oraz OpenWeather (Dane Meteorologiczne).
- Transformacja: W NiFi wykonywana jest konwersja danych z formatu JSON do docelowego formatu Parquet, ujednolicenie nazw pól oraz wstępne filtrowanie i dodawanie znaczników czasu.

II. Buforowanie Danych (Buffering)

- Narzędzie: Apache Kafka.
- Rola: Pełni rolę brokera wiadomości (Message Broker). Wstępnie przetworzone dane z NiFi są przekazywane do Kafki. Buforowanie to kluczowe dla obsługi dużej częstotliwości napływu danych (np. co 10 sekund z autobusów), zapewniając odporność na awarie i separację systemu (decoupling).

III. Warstwa Wsadowa (Batch Layer)

- Narzędzia: HDFS i Apache Hive.
- Rola: Przechowywanie surowych i przetworzonych danych historycznych oraz udostępnianie ich do złożonej analizy.
 - HDFS: Służy jako system plików do fizycznego składowania danych w formacie Parquet.
 - Apache Hive: Dane są ładowane do ustrukturyzowanych tabelarycznie baz danych w Hive. Tabele są partycjonowane (według daty i typu źródła) w celu optymalizacji zapytań przez Sparka.
- Wielkość Bloku Danych: Wielkość bloku danych w HDFS zostanie ustawiona na 128 MB. Zapewni to równowagę między optymalnym przetwarzaniem dużych plików przez Sparka a uniknięciem zbyt dużych narzutów na metadane.
- Stopień Replikacji: Stopień replikacji danych (Replication Factor) zostanie ustawiony na 3. Jest to standardowa konfiguracja, która gwarantuje wysoką dostępność i odporność danych na awarie węzłów w klastrze.

Przykładowa struktura katalogów:

Katalog HDFS	Właściciel	Uprawnienia	Format Danych	Referencja do Schematu
/user/bigdata/data/raw/	user_nifi	rwX-r--r--	JSON	Dane źródłowe (API)
/user/bigdata/data/parquet/	user_spark	rwXr-Xr-X	Parquet	Schemat 1: Dane źródłowe
/user/bigdata/data/hive/open_aq/	user_hive	rwXr-Xr-X	Parquet	Schemat 2: Jakość Powietrza
/user/bigdata/data/hive/ztm_buses/	user_hive	rwXr-Xr-X	Parquet	Schemat 3: Lokalizacje ZTM
/user/bigdata/data/hive/weather_data/	user_hive	rwXr-Xr-X	Parquet	Schemat 4: Dane Meteorologiczne

IV. Wsadowa Analiza Danych (Processing)

- Narzędzie: Apache Spark.
- Rola: Wykonywanie złożonych zadań analitycznych na dużej skali, operując na danych z Hive i HDFS.
- Kluczowe operacje:
 - Łączenie danych z różnych źródeł po kluczach wspólnych (Lokalizacja i Czas).
 - Filtrowanie i czyszczenie danych.
 - Obliczanie widoków wsadowych i agregatów (np. średnie opóźnienia autobusów w zależności od stężenia PM2.5 lub temperatury).

V. Warstwa Udostępniania (Serving Layer)

- Narzędzie: Apache HBase.
- Rola: Udostępnianie wyników analiz (widoków wsadowych) wygenerowanych przez Sparka.
 - HBase, jako baza danych NoSQL, umożliwia szybkie odczyty po kluczu, co jest niezbędne np. do zasilania interaktywnych paneli kontrolnych (dashboardów).

Uzasadnienie takiego wyboru narzędzi przy architekturze systemu:

- Apache NiFi: został wybrany ze względu na swoje graficzne środowisko do zarządzania przepływem danych oraz wbudowaną bibliotekę procesorów. Umożliwi to automatyczne oraz bez pisania kodu pobieranie danych z niestabilnych źródeł API oraz wizualne projektowanie złożonych transformacji (np. konwersji z JSON do Parquet).
- Apache Kafka: została włączona do architektury jako rozproszony broker wiadomości. Zapewnia to buforowanie danych przy wysokiej częstotliwości (np. dane ZTM, co 10 sekund) oraz działa jako izolacja systemów, chroniąc Warstwę Wsadową przed niestabilnością strumieni wejściowych.
- HDFS i Apache Hive: HDFS został wybrany jako podstawowy system składowania danych w warstwie wsadowej ze względu na jego horyzontalną skalowalność oraz odporność na awarie klastra. Główną rolą HDFS jest fizyczne przechowywanie dużych zbiorów surowych i przetworzonych danych historycznych pochodzących z wielu źródeł, w formacie Parquet. Taka architektura pozwala na efektywne składowanie danych (Batch Layer) oraz optymalne ich odczytywanie w operacjach wsadowych przez Apache Spark. Dzięki replikacji danych, system gwarantuje wysoką dostępność i spójność informacji. Apache Hive nakłada na te dane strukturę tabelaryczną. To połączenie umożliwia składowanie danych w zoptymalizowanym kolumnowym formacie Parquet i ułatwia ich późniejsze odpytywanie i przetwarzanie w trybie wsadowym przez Apache Spark.
- Apache Spark: to silnik do skalowalnego, wsadowego przetwarzania danych w pamięci (in-memory processing). Został wybrany ze względu na swoją wydajność, która jest niezbędna do łączenia dużych zbiorów danych (Pogoda, Jakość Powietrza i ZTM) oraz wykonywania złożonej analizy korelacji między nimi.
- Apache HBase: to kolumnowa baza danych NoSQL, która jest zoptymalizowana pod kątem losowych odczytów i zapisów. Jej rola w projekcie polega na przechowywaniu wynikowych widoków wsadowych (agregatów i wskaźników analizy) w celu ich szybkiego udostępniania aplikacjom końcowym.

4. Źródła danych

	1. Jakość powietrza (OpenAQ API)	2. Bieżące lokalizacje autobusów komunikacji miejskiej	3. Dane meteorologiczne
Dostępność danych	Publiczne API OpenAQ (Wersja 3) (https://api.openaq.org/docs). Dostęp wymaga uwierzytelnienia za pomocą Klucza API przekazywanego w nagłówku X-API-Key, który jest uzyskiwany po rejestracji w serwisie OpenAQ.	Publiczne API Miasta (po rejestracji i uzyskaniu klucza): <a href="https://api.um.warszawa.pl/api/action/busestrams_get/?resource_id=f2e5503e927d-4ad3-9500-4ab9e55deb59&apikey=<API_KEY>">https://api.um.warszawa.pl/api/action/busestrams_get/?resource_id=f2e5503e927d-4ad3-9500-4ab9e55deb59&apikey=<API_KEY>	OpenWeather Weather API (https://openweathermap.org/current) - limit 60 wywołań na minutę i 1 mln na miesiąc. Wymagana rejestracja i otrzymanie klucza.
Format	JSON (zgodny ze standardami REST)	JSON (zgodny ze standardami REST)	JSON / XML / HTML
Zawartość	Historyczne i bieżące pomiary stężeń zanieczyszczeń powietrza (głównie PM2.5, PM10, CO, NO2) wraz z dokładnymi metadanymi dotyczącymi lokalizacji i czasu pomiaru	Pozycje bieżące autobusów: - Lat, Lon - współrzędne GPS - Time czas wysłania sygnału GPS - Lines - numer linii - Brigade - numer brygady - VehicleNumber - numer taborowy pojazdu	Udostępnia bieżące dane pogodowe, prognozy minutowe, godzinowe i dzienne oraz archiwalne informacje z ostatnich 46 lat. Zawiera także alerty meteorologiczne, opisowe podsumowania warunków pogodowych i funkcję asystenta opartego na sztucznej inteligencji.
Przykład pozyskania	<code>curl -X GET "https://api.openaq.org/v3/locations/10776" -H "X-API-Key: 0281745eb1ee589f7842c04e2f826793dfdb04df63583da9aa6285ac077393b7"</code>	<code>curl -X GET "https://api.um.warszawa.pl/api/action/busestrams_get/?resource_id=f2e5503e-927d-4ad3-9500-4ab9e55deb59&type=1&apikey=51b9e237-4a51-4cf0-920c-933428d34e39"</code>	<code>curl 'https://api.openweathermap.org/data/2.5/weather?lat=44.34&lon=10.99&appid={API key}'</code>

5. Batch Layer - Implementacja HDFS

Batch layer w projekcie realizuje przechowywanie oraz przetwarzanie danych wsadowych (historycznych) dotyczących lokalizacji autobusów. Podstawową technologią magazynowania danych jest HDFS, który zapewnia rozproszony system plików odporny na awarie oraz umożliwia składowanie dużych wolumenów danych w klastrze. Wybór HDFS wynika z jego skalowalności, możliwości równoległego dostępu do danych i integracji z narzędziami ekosystemu Big Data (np. Spark). Dane trafiają do HDFS w formie surowej oraz oczyszczonej, a następnie mogą być okresowo analizowane i agregowane w zadaniach wsadowych.

1. Jakość Powietrza (<https://api.openaq.org/docs>)

Atrybuty:

- location_id (INT)
- location_name (STRING)
- latitude (FLOAT)
- longitude (FLOAT)
- parameter (STRING) - typ zanieczyszczenia
- value (FLOAT) - zmierzona wartość stężenia
- datetimeLocal (DATE DD-MM-YYYY HH:MM:SS)

Przykładowy wpis CSV:

```
location_id,location_name,latitude,longitude,parameter,value,datetimeLocal
10776,"Warszawa, al. Niepodległości",52.219,21.011,PM25,28.5,2025-10-26 18:00:00
10776,"Warszawa, al. Niepodległości",52.219,21.011,PM10,48.1,2025-10-26 18:00:00
10123,"Sowińskiego 25",52.261,21.077,PM25,15.2,2025-10-26 19:00:00
```

2. Bieżące lokalizacje autobusów komunikacji miejskiej (<https://api.um.warszawa.pl>):

Dane partycjonowane według dnia i godziny co pozwala na łatwą analizę opóźnień wybranej przez nas godziny.

Format Danych: Parquet

Atrybuty:

- Lat (FLOAT)
- Lon (FLOAT)
- VehicleNumber (STRING)
- Time (BIG INT)
- Lines (STRING)
- Ingest Time (BIG INT)

Przykładowy wpis CSV:

```
Lines, Lon, VehicleNumber, Time, Lat, Ingest Time
"135", 21.042563, "1819", 1768069756895, 52.266168, 1768069769895
"157", 21.043375, "1833", 1768069769987, 52.266089, 1768069769732
"168", 21.0274685, "1835", 1768069769855, 52.208151, 1768069769590
```

3. Dane meteorologiczne (<https://openweathermap.org/current>):

Atrybuty:

- event_time BIGINT,
- weather_id INT,
- weather_main STRING,
- weather_desc STRING,
- temp DOUBLE,
- feels_like DOUBLE,
- pressure INT,
- humidity INT,
- wind_speed DOUBLE,
- rain_1h DOUBLE,
- snow_1h DOUBLE

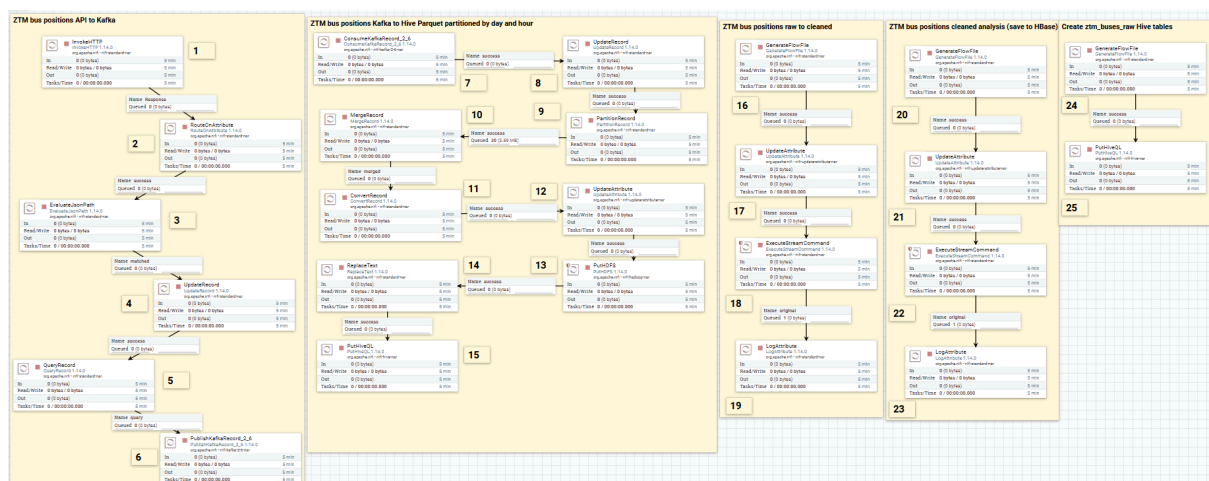
Przykładowy wpis CSV:

event_time,weather_id,weather_main,weather_desc,temp,feels_like,pressure,humidity,wind_speed,rain_1h,snow_1h 1768269257,701,Mist,mist,-8.46,-12.41,1017,90,2.06,0.0,0.0 1768269557,800,Clear,clear sky,-5.12,-7.30,1021,72,1.20,0.0,0.0

6. Implementacja NIFI

Apache NiFi służy do automatycznego pobierania danych o pozycjach autobusów Warszawskich, jakości powietrza i danych meteorologicznych oraz przetwarzania ich w ustandaryzowanej formie. Przepływ danych obejmuje wykonywanie zapytań HTTP, rozdzielanie poprawnych i błędnych odpowiedzi oraz czyszczenie i wyciąganie potrzebnych pól z JSON. Następnie dane są łączone w większe paczki rekordów oraz zapisywane. Końcowo NiFi uruchamia kody sparkowe służące do analizy zgromadzonych danych. Dzięki NiFi cały proces działa cyklicznie i bezobsługowo, z możliwością monitorowania i debugowania na każdym etapie.

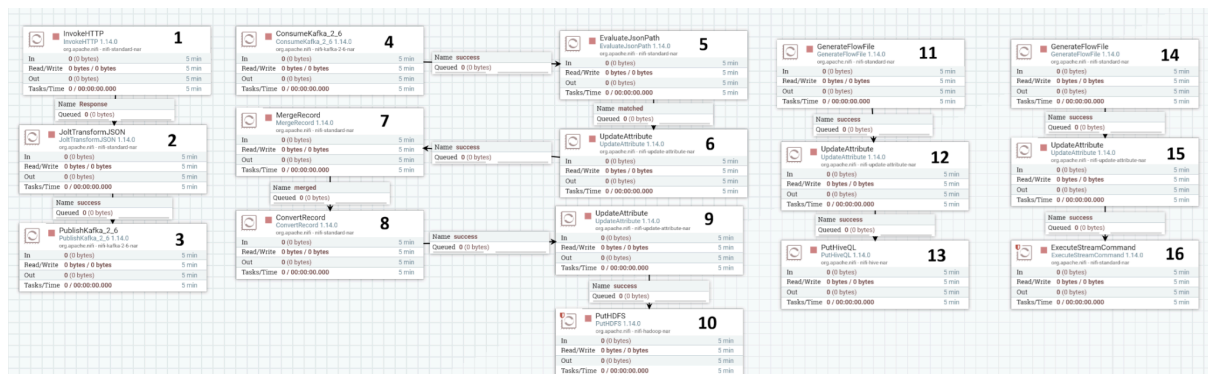
DANE ZTM_BUSES:



- 1) Wywołanie zapytania GET do API ZTM Warszawa w celu pobrania danych o lokalizacji autobusów miejskich

- 2) Sprawdzenie kodu odpowiedzi HTTP i przekierowanie danych dalej do ścieżki success jeśli kod 200
- 3) Wyciąga z treści JSON wartość pola result i zapisuje ją jako atrybut FlowFile
- 4) Dodaje pole ingest_time i zmienia formatowanie pola time
- 5) Filtruje dane tak aby wyciągnąć tylko lokalizacje pojazdów które dodały swoją lokalizację w ciągu ostatnich 30 sekund
- 6) Dodanie rekordów z flow file do topicu ztm.buses w kafce
- 7) Pobieranie rekordów z kafki z topicu ztm.buses
- 8) Dodanie zmiennych dt i hr wyliczonych ze zmiennej time które w dalszej części posłużą do partycjonowania danych
- 9) Partycjonowanie danych na podstawie daty i godziny
- 10) Łączenie danych w jeden duży plik na podstawie partition path zbierając dane z ostatniej godziny
- 11) Konwersja Json do Parquet
- 12) Dodanie parametrów dt i hr które zostaną użyte podczas zapisu danych
- 13) Zapisanie danych do HDFS do odpowiednich partycji utworzonych na podstawie dt i hr
- 14) Przygotowanie kodu HiveQL który zostanie użyty do dodania partycji danych z HDFS do naszej tabeli Hive
- 15) Uruchomienie kodu HiveQL służącego do dodania nowej partycji do naszej tabeli Hive
- 16) Inicjalizacja flow file służącego do przetwarzania danych za pomocą kodu spark
- 17) Ustawienie atrybutu dt który definiuje z którego dnia będziemy oczyszczać dane
- 18) Wykonanie skryptu spark przetwarzającego i oczyszczającego dane z wybranego dnia
- 19) Wypisania w logach NiFi atrybutów i podstawowych informacji o flow file, żeby łatwiej debugować i sprawdzić co przepływa dalej w procesie
- 20) Inicjalizacja flow file służącego do analizy oczyszczonych danych za pomocą kodu spark
- 21) Utworzenie odpowiednich atrybutów które dalej zostaną przekazane do kodu spark (atrybuty te definiują dzień, godzinę, linię autobusu, tabelę, ścieżkę do pliku z przystankami autobusowymi)
- 22) Wykonanie skryptu spark analizującego dane z wybranego dnia, godziny oraz linii autobusowych
- 23) Wypisania w logach NiFi atrybutów i podstawowych informacji o flow file, żeby łatwiej debugować i sprawdzić co przepływa dalej w procesie
- 24) Inicjalizacja flow file służącego do utworzenia tabeli hive dla surowych danych i przygotowanie kodu hive służącego do tego
- 25) Wykonanie tego kodu tworzącego tabelę hive

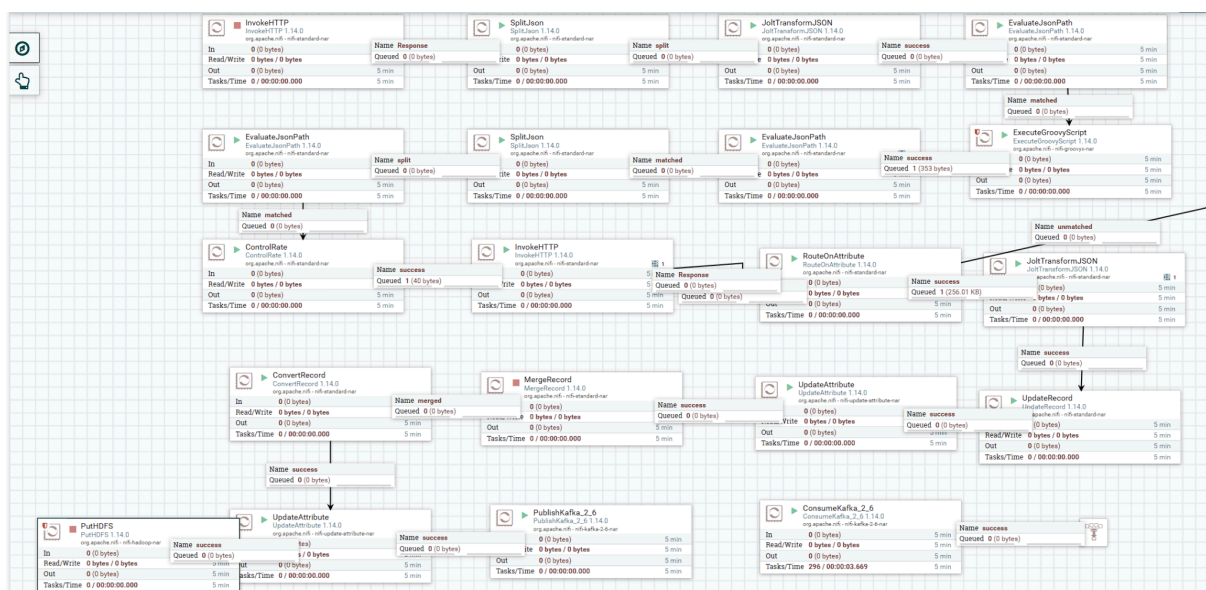
DANE WEATHER:



1. InvokeHTTP - pobiera dane pogodowe z OpenWeather API.
2. JoltTransformJSON - mapuje i spłaszcza surowy JSON do docelowej struktury pól.

- PublishKafka_2_6 - publikuje przetworzone dane do tematu Kafka.
- ConsumeKafka_2_6 - konsumuje dane pogodowe z Kafki do dalszego przetwarzania.
- EvaluateJsonPath - wyciąga wybrane pola z JSON-a do atrybutów FlowFile (np. event_time).
- UpdateAttribute (event_time → dt) - wylicza wartość partycji dt(YYYY-MM-DD) na podstawie czasu zdarzenia.
- MergeRecord - grupuje rekordy z tego samego dnia w większe batch'e.
- ConvertRecord - konwertuje dane z JSON do formatu Parquet.
- UpdateAttribute (ścieżka HDFS) - buduje docelową ścieżkę zapisu z partycją dt=YYYY-MM-DD.
- PutHDFS - zapisuje pliki Parquet do HDFS w katalogu partycji.
- GenerateFlowFile (Hive trigger) - wyzwala proces rejestracji partycji w Hive (CRON).
- UpdateAttribute (date dla Hive) - ustawia wartość dtużywaną w zapytaniu Hive.
- PutHiveQL - wykonuje ALTER TABLE ADD PARTITION w Hive Metastore.
- GenerateFlowFile (Spark trigger) - wyzwala uruchomienie joba Spark (CRON lub ręcznie).
- UpdateAttribute (dt dla Sparka) - przygotowuje parametr --dt przekazywany do joba Spark.
- ExecuteStreamCommand - uruchamia spark-submit, który czyta dane z Hive, liczy statystyki i zapisuje wyniki do HBase.

DANE OPEN_AQ:



- InvokeHTTP - Wywołanie zapytania GET do API OpenAQ v3 w celu pobrania listy lokalizacji pomiarowych w Warszawie (bbox: 20.851,52.097,21.271,52.368) z użyciem klucza API
- SplitJson - Podział odpowiedzi JSON na pojedyncze rekordy z tablicy \$.results, gdzie każdy rekord reprezentuje jedną lokalizację pomiarową
- JoltTransformJSON - Transformacja struktury JSON w celu wyciągnięcia kluczowych informacji o lokalizacji: location_id, location_name, współrzędnych geograficznych (latitude, longitude) oraz listy sensorów z ich parametrami

4. EvaluateJsonPath - Wyciągnięcie z treści JSON wartości pól location_id, location_name, latitude i longitude i zapisanie ich jako atrybuty FlowFile
5. ExecuteGroovyScript - Skrypt Groovy generujący zakres dat dla zapytań do API: data początkowa (stała: 2025-12-01) oraz data końcowa (wczorajsza data, ponieważ dane daily są dostępne za poprzedni dzień)
6. EvaluateJsonPath - Wyciągnięcie tablicy sensors z treści JSON i zapisanie jej jako nowej zawartości FlowFile
7. SplitJson - Podział tablicy sensorów na pojedyncze rekordy, gdzie każdy rekord reprezentuje jeden sensor
8. EvaluateJsonPath - Wyciągnięcie z treści JSON wartości pól sensor_id i parameter_name (nazwa mierzonego parametru, np. PM2.5, PM10) i zapisanie ich jako atrybuty FlowFile
9. ControlRate - Ograniczenie szybkości przetwarzania do 1 FlowFile na sekundę, aby nie przekroczyć limitów API OpenAQ (rate limiting)
10. InvokeHTTP - Wywołanie zapytania GET do API OpenAQ v3 w celu pobrania danych pomiarowych dziennych (days endpoint) dla konkretnego sensora w określonym zakresie dat (używając atrybutów sensor_id, date_start, date_end)
11. RouteOnAttribute - Sprawdzenie kodu odpowiedzi HTTP (invokehttp.status.code) i przekierowanie danych do ścieżki success jeśli kod równa się 200, w przeciwnym razie do unmatched
12. LogAttribute - Wypisanie w logach NiFi atrybutów i podstawowych informacji o FlowFile dla przypadków unmatched (błędne odpowiedzi API), aby ułatwić debugowanie
13. JoltTransformJSON - Transformacja struktury JSON wyników pomiarów: wyciągnięcie daty lokalnej pomiaru, wartości pomiaru oraz wstrzyknięcie atrybutów lokalizacji (location_id, location_name, latitude, longitude, parameter_name) do każdego rekordu, z konwersją typów na liczby
14. UpdateRecord - Zmiana formatowania pola datetimeLocal z formatu ISO 8601 (YYYY-MM-DDTHH:mm:ss) na format DD-MM-YYYY HH:mm:ss przy użyciu RecordPath i wyrażenia regularnego
15. UpdateAttribute - Dodanie atrybutu merge_group o wartości "all" w celu grupowania wszystkich rekordów razem podczas kolejnego kroku łączenia
16. MergeRecord - Łączenie wielu małych FlowFiles w jeden większy plik na podstawie atrybutu merge_group (Bin-Packing Algorithm), zbierając rekordy przez maksymalnie 2 minuty lub do osiągnięcia 100000 rekordów
17. ConvertRecord - Konwersja formatu danych z JSON do Parquet z użyciem JsonTreeReader (ze zdefiniowanym schematem Avro) i ParquetRecordSetWriter
18. UpdateAttribute - Dodanie atrybutów filename (z wzorcem air_quality_{timestamp}_{UUID}.parquet) i mime.type (application/parquet) przygotowujących plik do zapisu
19. PutHDFS - Zapisanie pliku Parquet do HDFS w katalogu /user/bigdata/data/parquet/open_aq ze strategią konfliktów "replace" (nadpisywanie istniejących plików)
20. PublishKafka_2_6 - Publikowanie przetworzonych danych JSON do topicu Kafka "open_aq_topic" na localhost:9092 bez potwierdzenia zapisu (acks=0)
21. ConsumeKafka_2_6 - Konsumowanie wiadomości z topicu Kafka "open_aq_topic" przez grupę konsumentów "nifi-consumer-new-1" z odczytem od najwcześniejszego dostępnego offsetu (earliest), pobierając maksymalnie 10000 rekordów na poll
22. Funnel - Punkt zbiorczy łączący strumienie danych z procesora ConsumeKafka do monitorowania

7. Implementacja Kafka

DANE ZTM_BUSES:

Dane o autobusach ZTM są publikowane do topica **ztm.buses** z wykorzystaniem procesora **PublishKafka_2_6** w Apache NiFi. Następnie, w osobnym przepływie NiFi, dane są pobierane z tego samego topica przez procesor **ConsumeKafka_2_6** i kierowane do dalszego przetwarzania. Wykorzystanie Apache Kafka umożliwia rozdzielenie etapu pozyskiwania danych od ich konsumpcji, zapewniając trwałość przesyłanych komunikatów oraz kontrolę offsetów. Separacja publikacji i konsumpcji w dwóch flow upraszcza utrzymanie rozwiązania oraz zwiększa elastyczność pipeline'u dla strumienia **ztm_buses**.

DANE WEATHER:

Dane pogodowe są publikowane do topica `api.openweather` z wykorzystaniem procesora **PublishKafka_2_6** w Apache NiFi, a następnie konsumowane z tego samego topica przez procesor **ConsumeKafka_2_6**. Zastosowanie Kafki pozwala na rozdzielenie etapu pobierania danych od ich dalszego przetwarzania, przy jednoczesnym zachowaniu trwałości i kontroli offsetów. Cały proces publikacji i konsumpcji danych realizowany jest w ramach jednego przepływu NiFi, co upraszcza zarządzanie i integrację pipeline'u.

DANE OPEN_AQ:

Kafka została zaimplementowana w NiFi:

1. **PublishKafka_2_6** - Publikowanie danych do topicu Kafka "open_aq_topic". Format wymiany: dane przesyłane są w formacie JSON jako tekstowe wiadomości Kafka, bez kompresji (`compression.type=none`), z kodowaniem UTF-8. Maksymalny rozmiar pojedynczej wiadomości to 100 MB. Brak potwierdzenia zapisu (`acks=0`) dla maksymalnej wydajności. Dane nie zawierają klucza partycjonowania, więc są dystrybuowane przez domyślny partycjoner Kafki (`DefaultPartitioner`) równomiernie pomiędzy partycje topicu.
2. **ConsumeKafka_2_6** - Konsumowanie wiadomości z topicu Kafka "open_aq_topic". Format przechowywania i odczytu: wiadomości są odczytywane w formacie tekstowym JSON z kodowaniem UTF-8, maksymalnie 10000 rekordów na pojedyncze wywołanie poll. Grupa konsumentów "nifi-consumer-new-1" śledzi offsety odczytu, umożliwiając wznowienie przetwarzania od ostatnio przetworzonej wiadomości. Strategia `auto.offset.reset` ustawiona na "earliest" zapewnia odczyt wszystkich dostępnych danych od początku topicu przy pierwszym uruchomieniu. Konsument honoruje transakcje Kafki (`honor-transactions=true`) dla zapewnienia spójności danych. Nagłówki wiadomości Kafka są dekodowane jako UTF-8 i przekazywane jako atrybuty `FlowFile`.

8. Implementacja Spark - uruchamiane zadania

1. Wczytywanie i parsowanie danych

- Odczyt plików CSV z danymi ZTM (transport publiczny)
- Odczyt plików parquet z danymi o jakości powietrza (smog)
- Odczyt plików parquet z danymi o pogodzie

2. Czyszczenie i transformacja danych

- Filtrowanie nieprawidłowych lub brakujących wartości
- Konwersja typów danych (stringi na liczby, daty)
- Obsługa null-i i wartości pustych
- Standaryzacja formatów dat

3. Agregacje i grupowania

- Obliczanie średnich wartości pogodowych (temperatura, ciśnienie, wilgotność, wiatr)
- Grupowanie danych smogowych według lokalizacji i parametrów
- Agregacja danych ZTM według linii i godzin
- Sumowanie liczby kursów

4. Filtrowanie i selekcja

- Wybór konkretnych dat (2026-01-10)
- Wybór konkretnej godziny (19:00)
- Filtrowanie według parametrów (np. tylko PM2.5, tylko temperatura)
- Selekcja TOP N rekordów (TOP 10 linii)

5. Łączenie zbiorów danych (potencjalnie)

- Łączenie danych pogodowych z danymi ZTM według daty
- Potencjalne łączenie lokalizacji czujników smogowych z trasami linii

6. Obliczenia statystyczne

- Obliczanie korelacji (Pearsona) między zmiennymi
- Obliczanie median, średnich, odchyleń standardowych

7. Konwersja do Pandas

- Konwersja DataFrame Spark → Pandas (dla wizualizacji)
- Collect() - zbieranie wyników do driver node

9. Implementacja Hive

DANE ZTM_BUSES_RAW:

Dla danych surowych ZTM utworzono tabelę `ztm_buses_raw`.

- Typ tabeli: Tabela zewnętrzna (EXTERNAL TABLE) - metadane w Hive Metastore, dane w HDFS.
- Typowanie danych: Schemat tabeli narzuca jednoznaczne typy danych dla poszczególnych atrybutów. Informacje opisowe takie jak **Lines**, **VehicleNumber** oraz **Brigade** zapisano jako **STRING**. Współrzędne geograficzne **Lon** i **Lat** przechowywane są jako wartości zmiennoprzecinkowe (**DOUBLE**). Znaczniki czasu **Time** oraz **ingest_time** zapisano jako liczby całkowite (**BIGINT**), co umożliwia łatwe przetwarzanie danych czasowych.
- Partycjonowanie: Tabela partycjonowana po dacie i godzinie (dt i hr)

DANE ZTM_BUSES_CLEANED:

Dla danych po oczyszczeniu utworzono tabelę `ztm_buses_cleaned`.

- Typ tabeli: Tabela zewnętrzna (EXTERNAL TABLE) - metadane w Hive Metastore, dane w HDFS.
- Typowanie danych: Schemat tabeli wymusza jednoznaczne typy danych dostosowane do analiz. Pola identyfikacyjne **line**, **vehicle_number** oraz **brigade** zapisano jako **STRING**. Współrzędne geograficzne **lat** i **lon** przechowywane są jako wartości zmiennoprzecinkowe (**DOUBLE**). Znacznik czasu zdarzenia **event_ts** ma typ **TIMESTAMP**, natomiast czas przetworzenia danych **ingest_ms** zapisano jako **BIGINT**. Dodatkowo tabela zawiera kolumny partycjonujące **dt** oraz **hr** typu **STRING**.
- Partycjonowanie: Tabela partycjonowana po dacie dt (PARTITIONED BY dt).

DANE WEATHER:

Dla danych pogodowych zdefiniowano tabelę `weather_data` o następującej charakterystyce:

- Typ tabeli: Utworzono tabelę zewnętrzną (EXTERNAL TABLE). Metadane tabeli są przechowywane w Hive Metastore, natomiast same dane znajdują się w HDFS pod wskazaną lokalizacją.
- Typowanie danych: Schemat tabeli narzuca jednoznaczne typy danych dla poszczególnych atrybutów pogodowych. Znaczniki czasu zdarzeń (**event_time**) zapisane są jako liczby całkowite (**BIGINT**), identyfikatory i parametry dyskretne, takie jak **weather_id**, **pressure** czy **humidity**, jako **INT**. Opisy tekstowe warunków pogodowych (**weather_main**, **weather_desc**) mają typ **STRING**, natomiast wartości ciągłe, takie jak

temperatura (temp, feels_like), prędkość wiatru (wind_speed) czy opady (rain_1h, snow_1h), zapisano jako DOUBLE.

- Partycjonowanie: Tabela jest partycjonowana po kolumnie dt (PARTITIONED BY dt), reprezentującej datę.

DANE OPEN_AQ:

Apache Hive nadaje danym OpenAQ strukturę relacyjną, umożliwiającą wykonywanie zapytań SQL. Zdefiniowano tabelę air_quality o następującej charakterystyce:

- Typ tabeli: Utworzono tabelę zewnętrzną (EXTERNAL TABLE). Metadane tabeli są przechowywane w Hive Metastore, natomiast dane pozostają bezpieczne na HDFS nawet w przypadku usunięcia definicji tabeli (DROP TABLE).
- Typowanie danych: Schemat tabeli wymusza konkretne typy danych dla parametrów środowiskowych: location_name i parameter jako ciągi znaków (STRING), współrzędne geograficzne (latitude, longitude) oraz wartości pomiarów (value) jako liczby zmiennoprzecinkowe (DOUBLE).
- Partycjonowanie: Dane są partycjonowane po dacie (PARTITIONED BY date). Jest to optymalizacja krytyczna dla szeregów czasowych - zapytania analizujące smog w konkretnym dniu. W tym wypadku jest to jedna partycja (dzień zebrania danych), gdyż danych jest mało i tworzenie wielu małych plików byłoby nieoptymalne w HDFS.

10. Implementacja HBase

DANE ZTM_BUSES:

Do serwowania zagregowanych statystyk opóźnień autobusów ZTM wykorzystano bazę NoSQL **HBase** (tabela **ztm_line_avg_delays**).

Źródło danych: Aplikacja Spark generuje podsumowania godzinowe dla linii autobusowych (średnie opóźnienie oraz liczbę pomiarów), a następnie zapisuje wyniki do HBase.

Klucz wiersza (Row Key): Zastosowano klucz złożony w formacie:

dt_hr_line (np. 2026-01-13_08_517)

co zapewnia naturalne uporządkowanie rekordów w czasie i umożliwia szybkie pobieranie danych dla konkretnej daty, godziny i linii.

Rodzina kolumn: Wykorzystano jedną rodzinę kolumn **cf**, przechowującą zarówno dane identyfikacyjne (line, dt, hour), jak i zagregowane metryki:

- **avg_time_diff_minutes** - średnie opóźnienie,
- **measurement_count** - liczba pomiarów,
- **total_time_diff** - suma opóźnień.

DANE WEATHER:

- Źródło danych: Aplikacja Spark wczytuje dane z tabeli Hive weather_data, filtrowane po dacie (dt), a następnie wylicza statystyki godzinowe (średnie wartości parametrów pogodowych oraz licznosc pomiarów).
- Klucz wiersza: Zastosowano klucz w formacie data_godzina (np. 2026-01-13_08), co zapewnia naturalne uporządkowanie rekordów w czasie i szybki dostęp do danych dla konkretnej godziny lub dnia.
- Rodzina kolumn: Wykorzystano jedną rodzinę kolumn cf, przechowującą zarówno informacje identyfikacyjne (data, godzina), jak i zagregowane metryki pogodowe (np. średnia temperatura, wilgotność, prędkość wiatru, ciśnienie).

DANE OPEN_AQ:

Do serwowania przetworzonych i zagregowanych metryk jakości powietrza wykorzystano bazę NoSQL HBase (tabela air_quality_metrics). Dane są tu przechowywane w modelu *Wide-Column Store*.

- Klucz Wiersza (Row Key): Zastosowano klucz złożony w formacie: NazwaStacji#DataPomiaru (np. Belgradzka#2025-12-01).
 - Taka konstrukcja klucza zapewnia fizyczne zgrupowanie wszystkich historycznych pomiarów dla danej stacji w jednym regionie bazy, co umożliwia błyskawiczne pobieranie trendów zmian jakości powietrza dla konkretnej lokalizacji.
- Rodziny Kolumn (Column Families): Dane podzielono na dwie rodziny:
 - meta: Przechowuje atrybuty informacyjne, takie jak nazwa lokalizacji (meta:location), data (meta:date) oraz znacznik czasu przetworzenia (meta:processed_at).
 - stats: Przechowuje właściwe wyniki pomiarów dla różnych parametrów, dynamicznie dodawane jako kolumny (np. stats:pm25, stats:pm1, stats:temperature, stats:relativehumidity).
- Serializacja i wymiana: Aplikacja Spark dokonuje transformacji danych do formatu bajtowego (wymaganego przez HBase). Wartości liczbowe i tekstowe są kodowane w standardzie UTF-8 przed wysłaniem do bazy (np. str(value).encode('utf-8')).

11. Dane docelowe (Widoki wsadowe)

Dane docelowe stanowią skonsolidowane i przetworzone wyniki analizy korelacyjnej wykonanej przez Apache Spark. Dane te zostaną zapisane w formacie Parquet w Apache Hive (do celów archiwalnych i dalszej analizy historycznej) oraz w Apache HBase (do szybkiego udostępniania i losowych odczytów w Warstwie Udostępniania). Widoki wsadowe zawierają kluczowe wskaźniki potrzebne do podjęcia strategicznych decyzji przez Zarząd Transportu Miejskiego (ZTM).

Przykładowa tabela atrybutów wynikowych:

Atrybut	Typ Danych	Opis
ROWKEY	STRING	Klucz złożony: Data_Godzina + ID_Regionu/Linia (służy do szybkiego odczytu w HBase).
Timestamp (Agregacji)	DATETIME	Czas, do którego odnosi się agregacja danych.
ID_Linii	STRING	Numer linii autobusowej, dla której obliczono opóźnienie.
Średnie_Opóźnienie [min]	FLOAT	Kluczowy wskaźnik: obliczona średnia różnica w czasie dojazdu w danej godzinie.
Średnia_Temp [K]	FLOAT	Średnia temperatura powietrza w czasie opóźnienia.
Max_PM25_Stężenie	FLOAT	Maksymalna wartość PM2.5 (Jakość Powietrza) zarejestrowana w regionie.
Warunki_Pogodowe	STRING	Opisowe podsumowanie warunków pogodowych (np. "Opady Śniegu", "Słonecznie").
Liczba_Kursów	INT	Liczba kursów uwzględnionych w agregacji.

12. Testy Funkcjonalne

Poniższe testy przedstawiają uproszczony plan weryfikacji działania kluczowych komponentów systemu Big Data zbudowanego w oparciu o Apache NiFi, Hive, Spark i HBase. Testy mają na celu potwierdzenie poprawnego przepływu danych, ich składowania oraz analizy.

Komponent	Cel testu	Kroki	Oczekiwany wynik
Apache NiFi	Sprawdzić, czy dane z API są poprawnie pobierane i zapisywane do pliku.	Uruchomić przepływ danych w NiFi. Sprawdzić, czy procesory zakończyły się sukcesem.	Plik z danymi w formacie CSV/JSON zapisany w katalogu wyjściowym.
Apache Hive / HDFS	Sprawdzić, czy dane są zapisane w HDFS i dostępne w Hive.	Załadować pliki do HDFS, wykonać proste zapytanie <code>SELECT COUNT(*)</code> .	Zapytanie zwraca liczbę rekordów większą niż 0.
HDFS (Podgląd Danych)	Sprawdzić, czy dane w formacie Parquet są poprawnie zapisane i możliwe do odczytu.	Wykonać komendę <code>hdfs dfs -cat [przykładowy_plik_parquet] head -n 10</code>	Wyświetlenie pierwszych 10 rekordów z pliku. Potwierdzenie struktury danych i formatu.
HDFS (Weryfikacja Replikacji)	Sprawdzić, czy pliki w HDFS mają zadeklarowany stopień replikacji (np. 3).	Wykonać komendę <code>hdfs fsck [ścieżka_katalogu] -files -blocks -locations</code> .	Wyświetlenie szczegółów pliku, w tym sekcji: Replication factor: 3/3 (lub innej zadeklarowanej wartości).

Apache Spark	Sprawdzić, czy Spark przetwarza dane i tworzy wyniki.	Uruchomić skrypt Spark (ETL lub analiza). Sprawdzić wynikowy plik.	Powstaje plik z wynikami w formacie CSV/Parquet.
Apache HBase	Sprawdzić, czy dane można odczytać po kluczu.	Wykonać polecenie get w HBase Shell dla przykładowego klucza.	Zwracany jest rekord z wartościami analiz.
Apache Kafka	Sprawdzić, czy dane są poprawnie publikowane przez NiFi i dostępne w buforze wiadomości.	1. Uruchomić przepływ danych w NiFi. 2. Uruchomić konsumenta Kafki (np. kafka-console-consumer) dla docelowego tematu (topic).	Konsumer odbiera i wyświetla strumień wiadomości (np. w formacie JSON lub Parquet), a liczba wiadomości w temacie wzrasta.

Testy mogą zostać potwierdzone zrzutem ekranu lub logiem pokazującym poprawne działanie danego komponentu. Ze względu na jakość raportu zostaną pokazane tu tylko niektóre testy - wszystkie działają, niektóre zostały zaprezentowane na zajęciach, a także w plikach ipynb oraz py.

Struktura katalogów:

Browse Directory

/user/bigdata/data/parquet							Go!
Permission	Owner	Group	Size	Last Modified	Replication	Block Size	Name
drwxr-xr-x	vagrant	supergroup	0 B	14.01.2026, 13:27:26	0	0 B	open_aq
drwxr-xr-x	vagrant	supergroup	0 B	14.01.2026, 13:03:26	0	0 B	weather_data
drwxr-xr-x	root	supergroup	0 B	14.01.2026, 13:14:15	0	0 B	ztm_buses_cleaned
drwxr-xr-x	hive	supergroup	0 B	14.01.2026, 13:08:06	0	0 B	ztm_buses_raw

/user/bigdata/data/parquet/ztm_buses_cleaned

Go!

Permission	Owner	Group	Size	Last Modified	Replication	Block Size	Name
drwxr-xr-x	root	supergroup	0 B	11.01.2026, 15:41:41	0	0 B	dt=20260110

/user/bigdata/data/parquet/ztm_buses_cleaned/dt=20260110

Go!

Permission	Owner	Group	Size	Last Modified	Replication	Block Size	Name
drwxr-xr-x	root	supergroup	0 B	11.01.2026, 15:41:38	0	0 B	hr=18
drwxr-xr-x	root	supergroup	0 B	11.01.2026, 15:41:39	0	0 B	hr=19
drwxr-xr-x	root	supergroup	0 B	11.01.2026, 15:41:40	0	0 B	hr=20
drwxr-xr-x	root	supergroup	0 B	11.01.2026, 15:41:41	0	0 B	hr=21
drwxr-xr-x	root	supergroup	0 B	11.01.2026, 15:41:41	0	0 B	hr=22

/user/bigdata/data/parquet/ztm_buses_raw

Go!

Permission	Owner	Group	Size	Last Modified	Replication	Block Size	Name
drwxr-xr-x	root	supergroup	0 B	11.01.2026, 08:55:12	0	0 B	dt=20260110
drwxr-xr-x	root	supergroup	0 B	11.01.2026, 15:00:26	0	0 B	dt=20260111

/user/bigdata/data/parquet/ztm_buses_raw/dt=20260110

Go!

Permission	Owner	Group	Size	Last Modified	Replication	Block Size	Name
drwxr-xr-x	root	supergroup	0 B	10.01.2026, 18:39:07	0	0 B	hr=09
drwxr-xr-x	root	supergroup	0 B	10.01.2026, 19:01:30	0	0 B	hr=18
drwxr-xr-x	root	supergroup	0 B	10.01.2026, 20:33:58	0	0 B	hr=19
drwxr-xr-x	root	supergroup	0 B	10.01.2026, 22:00:39	0	0 B	hr=20
drwxr-xr-x	root	supergroup	0 B	10.01.2026, 23:00:37	0	0 B	hr=21
drwxr-xr-x	root	supergroup	0 B	11.01.2026, 08:55:11	0	0 B	hr=22
drwxr-xr-x	root	supergroup	0 B	11.01.2026, 09:06:44	0	0 B	hr=23

Podgląd Danych:

Lokalizacje autobusów miejskich:

Aby podejrzeć dane dotyczące lokalizacji autobusów miejskich dla wybranej daty i godziny należy wykonać następujące komendy:

spark-shell

```
val df = spark.read.parquet("hdfs:///user/bigdata/data/parquet/ztm_buses_raw/dt=20260110/hr=09/")
```

```
df.show(false)
```

W wybranej ścieżce `hdfs:///user/bigdata/data/parquet/ztm_buses_raw/dt=20260110/hr=09/` możemy modyfikować `dt` oraz `hr` a także możemy podjąć dane surowe `ztm_buses_raw` lub oczyszczone `ztm_buses_cleaned`

```
vagrant@node1:~$ spark-shell
26/01/13 07:12:26 WARN util.Utils: Your hostname, node1 resolves to a loopback address: 127.0.0.1; using 10.0.2.15 instead (on interface enp0s3)
26/01/13 07:12:26 WARN util.Utils: Set SPARK_LOCAL_IP if you need to bind to another address
26/01/13 07:12:26 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
Spark context Web UI available at http://10.0.2.15:4040
Spark context available as 'sc' (master = local[*], app id = local-1768288355143).
Spark session available as 'spark'.
Welcome to

      ____
     / ___/
    / __/
   /___/
  version 3.1.2

Using Scala version 2.12.10 (OpenJDK 64-Bit Server VM, Java 1.8.0_292)
Type in expressions to have them evaluated.
Type :help for more information.

scala> val df = spark.read.parquet("hdfs://user/bigdata/data/parquet/ztm_buses_raw/dt=20260110/hr=09/")
df: org.apache.spark.sql.DataFrame = [Lat: double, Lon: double ... 5 more fields]

scala> df.show(10, false)
+-----+-----+-----+-----+-----+-----+-----+
|Lat|Lon|VehicleNumber|Lines|Brigade|Time|ingest_time|
+-----+-----+-----+-----+-----+-----+-----+
|52.2035485|21.07097|1803|119|2|1768035151000|1768035152672|
|52.2002491|21.0236035|1804|119|5|1768035152000|1768035152672|
|52.1866791|21.2248056|1806|161|1|1768035154000|1768035152672|
|52.186098|21.1434113|1807|219|3|17680351541000|1768035152672|
|52.397891|21.0378556|18072|L-8|M1|1768035154000|1768035152672|
|52.227808|21.1026648|1808|219|2|1768035154000|1768035152672|
|52.2661105|21.0903393|1809|119|1|1768035153000|1768035152672|
|52.172998|21.177201|1810|161|2|17680351544000|1768035152672|
|52.258943|21.178123|1811|196|1|17680351541000|1768035152672|
|52.214226|21.133769|1812|219|1|1768035144000|1768035152672|
+-----+-----+-----+-----+-----+-----+-----+
only showing top 10 rows

scala>
```

Weryfikacja Replikacji:

Browse Directory

/user/bigdata/data/parquet/open_aq/date=2026-01-13	Go!
--	-----

Permission	Owner	Group	Size	Last Modified	Replication	Block Size	Name
-rw-r--r--	root	supergroup	237.59 KB	13.01.2026, 22:10:00	3	128 MB	air_quality_2026-01-13_210510_69fc3fc4-e634-44ea-a872-cb8bfdb2c4f0.parquet

13. Podział prac

Poniżej znajduje się tabela ze wstępnym podziałem pracy, który może ulec zmianie i w takim przypadku zostanie poprawiony w finalnym raporcie projektu:

Zadanie projektowe	Jakub Kaproń (Ingestion)	Miłosz Kita (Analysis)	Michał Kukła (Batch/Serving)
Zaprojektowanie idei projektu	✓	✓	✓
Utrzymanie projektu na GitHub			✓
Dokumentacja Projektu (Raport)	✓	✓	✓
Pozyskanie danych (Konfiguracja API)	✓	✓	✓
Automatyzacja przepływu danych (Apache NiFi)	✓	✓	✓
Buforowanie danych (Apache Kafka)	✓	✓	
Składowanie danych (HDFS, Apache Hive - Batch Layer)		✓	✓
Składowanie agregacji NoSql (Apache HBase - Serving Layer)		✓	
Wsadowa analiza danych (Apache Spark)		✓	✓
Testy funkcjonalne	✓		